

SQLite-Carver-Pro

Technical Reference Manual & User Guide

Version 1.2.0 — Forensic Analysis Toolkit

⚠ Forensic Analysis Advisory & Mandatory Verification:

Binary carving and slack space recovery utilize heuristic pattern recognition to reconstruct partially overwritten or unreferenced database cells. **All carved artifacts, timestamps, and reconstructed rows must be independently verified, corroborated, and cross-referenced** with complementary system logs or primary sources before reaching definitive analytical, investigative, or legal conclusions. The software is provided "as is", without warranty.

Coverage & Capabilities:

- Low-level SQLite B-Tree parsing & O(1) carving algorithms
- Mathematical Confidence Threshold calculation (0.0 to 1.0)
- Freeblocks, Slack Space, Unallocated Margins & Freelist recovery
- Resilient text extraction for truncated cell fragments in Slack
- Strict Protobuf false-positive elimination & zlib/bplist decoding
- Interactive WAL Timeline & before/after transaction diffs (--table filter)
- Full Internationalization (English / French) & Unified Exporters

Author: Dam-FOR3K (with technical assistance from Antigravity / Google DeepMind)

Compatibility: Windows 64-bit / Linux / macOS (Python 3.11+ & Standalone .exe)

Document Date: August 2026 (v1.2.0 Release)

Table of Contents

1. Mandatory Verification Advisory & Legal Disclaimer

2. Introduction & Architecture Overview

3. SQLite Storage Internals & Forensic Carving Mechanisms

- 3.1. Physical anatomy of a SQLite database file (.db)
- 3.2. B-Tree page structures & Varint encoding
- 3.3. The 4 forensic carving zones (Exact Slack interval calculation)
- 3.4. Overflow chain reassembly
- 3.5. Write-Ahead Log (WAL) Journal & Automatic Carve merging
- 3.6. Automatic decoding of nested binary payloads (Apple bplist, Protobuf, zlib)

4. Understanding the Forensic Confidence Score (Confidence Threshold)

- 4.1. Why a confidence score?
- 4.2. Mathematical formula, SQLite invariants & Nested structure rating
- 4.3. Interpretation guide for the 4 confidence tiers
- 4.4. How to choose the right threshold for your investigative scenario

5. Command Line Interface & Standalone Executable Guide

- 5.1. Database inspection (`info`)
- 5.2. Deleted record carving & WAL detection (`carve`)
- 5.3. Recursive forensic search (`search`)
- 5.4. Transaction differential & timeline (`wal-diff` with table filter)
- 5.5. Standalone binary microscope with metadata (`decode-blob`)

6. Forensic Reports & Export Formats

- 6.1. The Dual-View Interactive HTML Dashboard (All Evidence & WAL Timeline with 60 FPS Pagination & Language Switcher)
- 6.2. Indented JSON (`.json`) & JSON Lines (`.jsonl`)
- 6.3. Excel Tabular CSV (`.csv`) & Apache Parquet (`.parquet`)

7. Python API for Automated Pipelines

1. Mandatory Verification Advisory & Legal Disclaimer

⚠ Essential Methodological Guideline

In any digital analysis, **no critical, investigative, or legal decisions should rely solely on automated heuristic reconstruction**. Carved data extracted from freeblocks, slack margins, or transaction logs may represent partial, overwritten, or historical fragments. Analysts must independently corroborate findings against primary evidence and operating system logs.

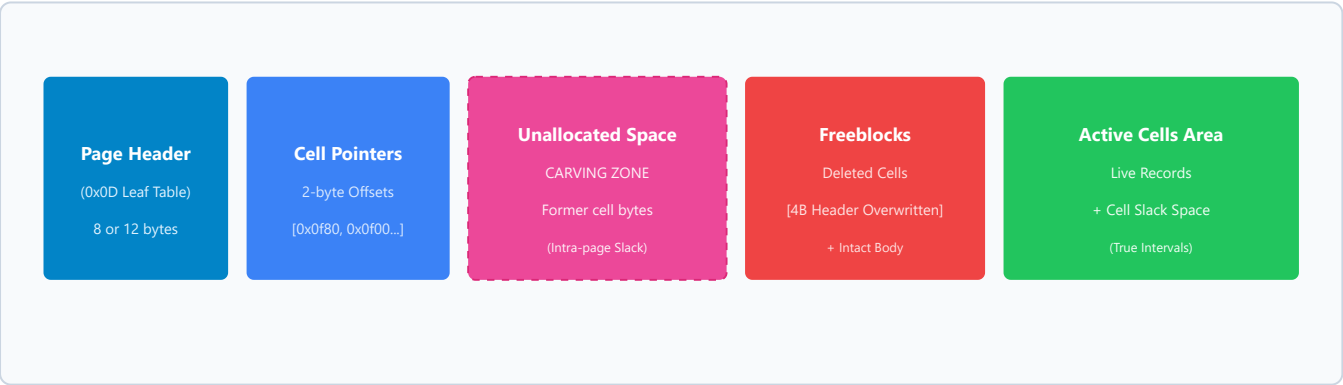
2. Introduction & Architecture Overview

In Digital Forensics and Incident Response, SQLite databases represent one of the most critical evidence repositories (Signal, WhatsApp, Telegram chat histories, Chrome/Edge/Firefox browser artifacts, iOS/Android system databases, authentication tokens, event logs).

Traditional carving utilities frequently rely on greedy text dumping, discarding relational schemas, column types, cell alignments, and complex binary serializations embedded inside BLOB columns.

SQLite-Carver-Pro v1.2.0 is a binary analysis toolkit capable of dissecting B-Tree structures, recovering deleted cell remnants in slack space and freeblocks, reassembling multi-page overflow chains, and automatically decoding nested binary payloads (Apple Binary Plist, Google Protocol Buffers, zlib/deflate compressed streams).

Visual Anatomy of a SQLite Leaf Page (4096B)



3. SQLite Storage Internals & Forensic Carving Mechanisms

3.1. Physical anatomy of a SQLite database file (.db)

A SQLite database is split into uniform pages (commonly 4096 bytes, or 512 to 65536 bytes). Page 1 begins with a 100-byte database header containing encryption flags, page sizes, and freelist trunk pointers.

3.2. B-Tree page structures & Varint encoding

SQLite records serialize column metadata using **Varints** (1 to 9 bytes variable-length integers in Big-Endian) and a table of **Serial Types**:

Serial Type	Data Type & Byte Length	Forensic Significance
0	0 bytes	NULL value.
1, 2, 3, 4, 5, 6	1, 2, 3, 4, 6, 8 bytes	Signed integers (with sign extension for 24-bit and 48-bit integers).
7	8 bytes	IEEE 754 floating-point number (64-bit float).
8, 9	0 bytes	Integer constants 0 or 1 (SQLite size optimization).
$N \geq 12$ (Even)	$(N - 12) / 2$ bytes	Raw binary payload BLOB (e.g. images, bplist, protobuf).
$N \geq 13$ (Odd)	$(N - 13) / 2$ bytes	UTF-8 encoded string TEXT .

4. Understanding the Forensic Confidence Score (Confidence Threshold)

4.1. Why a confidence score?

When examining unallocated disk clusters, freeblocks, or damaged databases, residual bytes can randomly match valid Varint sequences. Without strict validation, binary noise could be falsely presented as evidentiary rows (false positives).

To ensure **analytical reliability**, `sqlite-carver-pro` assigns each recovered record an objective **Confidence Score from 0.00 to 1.00**.

4.2. Mathematical Score Formula & Nested Structure Rating

The confidence score is computed dynamically across 5 strict forensic pillars:

1. **Physical Payload Invariant:**

The physical formula `payload_size = header_size + sum(column_lengths)` must hold true. Upon valid cell identification, the scanning cursor advances by the exact cell length to prevent duplicate ghost matches.

2. **Printable Character Sanity Checks:**

Every column typed as `TEXT` (odd serial type ≥ 13) is verified against UTF-8/ASCII printable character tables. If corrupted non-printable sequences are present, the score is severely penalized.

3. **Schema Heuristic Matching & Intrinsic Score:**

Recovered column types are matched against active table schemas in `sqlite_schema`. If no schema is available (e.g. dropped tables), a dynamic intrinsic score is derived from type coherence and readability (annotated as *unmapped schema* in details).

4. **Overwritten Header Penalty:**

In *freeblocks*, SQLite overwrites the first 4 bytes with its linked-list pointer. The carver reconstructs the cell from offset + 4 with a calibrated structural penalty (baseline `0.85`).

5. **Calibrated Embedded BLOB Rating:**

To prevent confidence inflation on signature-less formats:

- **Apple Binary Plist (`plist00`):** `1.00` (Strict magic header + certified object graph).
- **Zlib / Deflate Stream + Plaintext:** `0.95` (CRC/Adler32 verification + printable UTF-8).
- **Zlib + Protobuf or Binary:** `0.85` (Verified CRC decompression).
- **Google Protocol Buffers (Multi-field):** `0.85` (Validated wire types and field indices).
- **Google Protocol Buffers (Single-field / Isolated):** `0.65` (Conservative score to eliminate false positives).

4.3. Interpretation Guide for the 4 Confidence Tiers

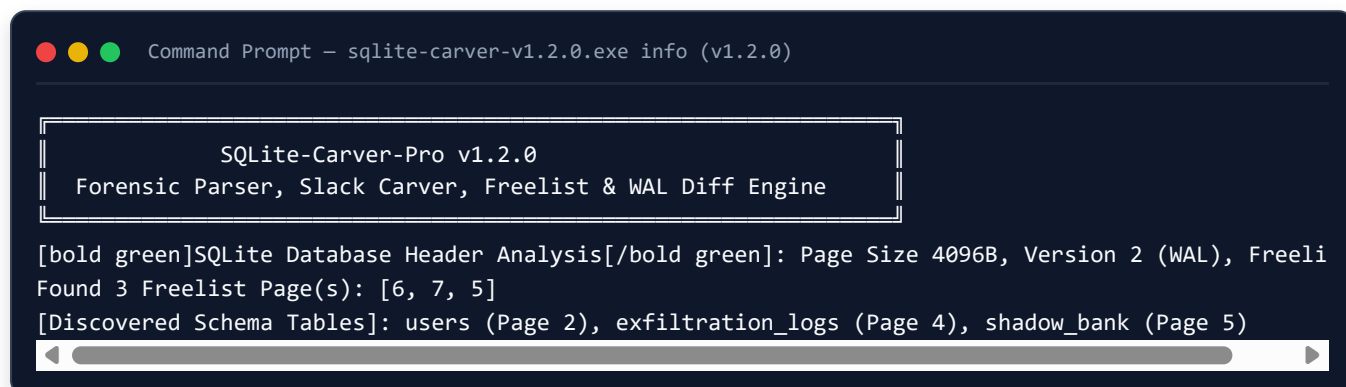


Score Range	Confidence Tier	Typical Origin & Interpretation
0.85 – 1.00	High Fidelity (Maximum Certainty)	Active cells or recently deleted intact records matching schema, 100% printable text.
0.65 – 0.84	Good Confidence (Reconstructed Rows)	Recovered from Freeblocks (overwritten 4-byte header), partial/NULL rows, or multi-field Protobuf payloads.
0.50 – 0.64	Moderate Confidence (Slack & Relics)	Cell fragments in Slack Space or orphaned Freelist Pages (tables dropped from <code>sqlite_schema</code>).
< 0.50	Binary Noise (Filtered by Default)	Fragmented artifacts, compressed residues, or encryption keys automatically discarded to avoid report flooding.

5. Command Line Interface & Standalone Executable Guide

5.1. Database inspection (info)

```
# Low-level diagnostic, freelist detection, and schema table listing (with language selection)
sqlite-carver-v1.2.0.exe info evidence.db --lang en
```



```
Command Prompt - sqlite-carver-v1.2.0.exe info (v1.2.0)

SQLite-Carver-Pro v1.2.0
Forensic Parser, Slack Carver, Freelist & WAL Diff Engine

[bold green]SQLite Database Header Analysis[/bold green]: Page Size 4096B, Version 2 (WAL), Freeli
Found 3 Freelist Page(s): [6, 7, 5]
[Discovered Schema Tables]: users (Page 2), exfiltration_logs (Page 4), shadow_bank (Page 5)
```

5.2. Deleted record carving & WAL detection (carve)

Automatic WAL Detection: The companion WAL journal is merged automatically by default. If the WAL file is damaged or unreadable, a clear warning is emitted to the console.

```
# 1. Complete carving (active + deleted + companion WAL journal)
sqlite-carver-v1.2.0.exe carve evidence.db

# 2. Isolate ONLY deleted records (Freeblocks, Freelist, Slack, WAL mutations)
sqlite-carver-v1.2.0.exe carve evidence.db --deleted-only

# 3. Filter by strict confidence threshold (e.g. >= 0.80 to exclude fragmented slack)
sqlite-carver-v1.2.0.exe carve evidence.db --min-confidence 0.80

# 4. Export full interactive web report with WAL Timeline tab
sqlite-carver-v1.2.0.exe carve evidence.db --export forensic_report.html
```

5.3. Recursive forensic search (search)

```
# Recursive search across active/deleted records, BLOBs, and WAL journal
sqlite-carver-v1.2.0.exe search evidence.db "session_token_xyz"

# Explicitly disable companion WAL inclusion
sqlite-carver-v1.2.0.exe search evidence.db "token" --no-include-wal

# Search hex byte sequences inside binary BLOB fields
sqlite-carver-v1.2.0.exe search evidence.db "deadbeef" --hex
```

5.4. Transaction differential & timeline (wal-diff)

```
# Reconstruct chronological transaction mutations
sqlite-carver-v1.2.0.exe wal-diff evidence.db evidence.db-wal

# Filter WAL diffs to a specific suspect table
sqlite-carver-v1.2.0.exe wal-diff evidence.db evidence.db-wal --table users --export diff_users.html
```


6. Forensic Reports & Export Formats

6.1. The Dual-View Interactive HTML Dashboard (All Evidence & WAL Timeline with 60 FPS Pagination & Language Switcher)

The HTML export produces a **100% Standalone Air-Gapped Forensic Dashboard** that opens directly in any browser (Edge, Chrome, Firefox) without requiring an internet connection or web server.

Ultra-Smooth 60 FPS Engine: In-memory pre-indexing and dynamic pagination (50, 100, 250, 500 rows, or All) ensure instantaneous responsiveness even on large databases containing tens of thousands of records.

```
sqlite-carver-v1.2.0.exe carve evidence.db --export forensic_report.html
```



6.2. Export Formats Reference

Format	Extension	Target Audience & Use Case
HTML Report	.html	Recommended for investigators. Dual-view interactive dashboard with instant search, confidence slider, 60 FPS pagination, WAL timeline tab, and decoded BLOB viewers.
Indented JSON	.json	Clean indented human-readable hierarchy for code editors (VS Code, Notepad++).
JSON Lines	.jsonl	Streaming single-line records for SIEM ingestion (Splunk, ElasticSearch, Cortex XSOAR).
CSV Spreadsheet	.csv	Tabular analysis in Microsoft Excel or external SQL ingestion (includes forensic details column).
Apache Parquet	.parquet	High-performance columnar compressed binary for forensic data science (Polars, DuckDB, Pandas).

7. Python API for Automated Pipelines

```
from pathlib import Path
from sqlite_carver import SQLiteCarver, WalDiffEngine, ForensicSearchEngine, export_html

# 1. Programmatic carving from raw bytes (v1.2.0)
raw_db = Path("evidence.db").read_bytes()
carver = SQLiteCarver(raw_db)

# 2. Extract records filtered by confidence (e.g. >= 0.80)
records = [r for r in carver.carve_all() if r.confidence >= 0.80]
for r in records:
    if r.source != "active":
        print(f"[{r.source.upper()}] Page {r.page_id} | Table: {r.matched_table} | Conf: {r.confidence}")

# 3. Recursive evidence search
engine = ForensicSearchEngine(raw_db)
matches = engine.search("session_token_xyz")
print(f"Found {len(matches)} token occurrences!")

# 4. Generate interactive bilingual HTML report
export_html(records, "automated_report.html")
```

Recommended DFIR Best Practices

1. Always preserve original evidence integrity by analyzing write-blocked disk images or bit-stream copies.
2. Keep companion `.db-wal` and `.db-shm` files in the same directory for comprehensive transaction timeline diffs.
3. Utilize the self-contained `.html` report format for legal deliverables: it empowers analysts and non-technical stakeholders to interactively inspect evidence.